LABORATORIO 09

Autenticación JWT y autorización por roles

INTEGRANTE	CURSO	FECHA
Jesus Castillo Sumire	Desarrollo de Aplicaciones Web	19 de octubre de 2026

Introducción y objetivo

El objetivo de este laboratorio es construir una API REST segura que registre usuarios, autentique credenciales y emita tokens JWT. En cada petición protegida, el token identifica al usuario y Spring Security comprueba si sus roles permiten acceder al recurso.

Cómo se trabajó y cómo se logró

Se creó un proyecto Spring Boot independiente, se conectó a MySQL de XAMPP y se implementaron las entidades y repositorios de usuarios y roles. Luego se incorporaron el registro público limitado a USER, el inicio de sesión, la firma y validación del JWT, el filtro Bearer y la autorización para USER, ADMIN y MODERATOR. La colección de Postman documenta las solicitudes de verificación.

Herramientas y conexión

Visual Studio Code, Java 17, Spring Boot 3.4.5, Spring Data JPA/Hibernate, Spring Security, Bean Validation, Maven Wrapper, MySQL Connector/J, JJWT 0.11.5 y Postman. XAMPP provee MariaDB en localhost:3306; la aplicación Java se conecta directamente a security_db y escucha en el puerto 8080. Apache solo se usa para phpMyAdmin.

Flujo y procedimiento

Postman → POST /api/auth/signin → AuthenticationManager y BCrypt → JWT firmado → Authorization: Bearer → JwtAuthFilter → permiso por rol → endpoint
XAMPP MySQL → security_db → Hibernate crea/actualiza roles, users y user_roles → DataInitializer carga roles y cuentas iniciales si faltan

Para ejecutar: iniciar MySQL en XAMPP, abrir el proyecto lab09-jwt-security en Visual Studio Code, revisar las credenciales de application.properties y ejecutar .\mvnw.cmd spring-boot:run. createDatabaseIfNotExist=true solicita crear security_db al conectarse si el usuario MySQL tiene permiso; Hibernate gestiona las tablas al iniciar.

Alcance implementado y base de datos

La solución usa tres tablas. USERS contiene la cuenta y el hash BCrypt; ROLES define USER, ADMIN y MODERATOR; USER_ROLES enlaza usuarios con roles mediante una relación muchos a muchos. Las claves únicas impiden duplicar username, email o la asociación usuario/rol.

Tabla	Campos principales	Función
users	id, username, email, password	Identidad de usuario y contraseña cifrada con BCrypt.
roles	id, name	Catálogo de permisos disponibles.
user_roles	user_id, role_id	Tabla intermedia con claves foráneas hacia users y roles.

Modelo de relaciones

USERS	USER_ROLES	ROLES
id (PK) username (UNIQUE) email (UNIQUE) password (BCrypt hash)	user_id (PK, FK) role_id (PK, FK)	id (PK) name (UNIQUE)

Relaciones: USERS 1 → N USER_ROLES; ROLES 1 → N USER_ROLES. En conjunto, USERS y ROLES mantienen una asociación muchos a muchos.

La clave compuesta de user_roles evita registrar dos veces el mismo rol para una cuenta. El script database/create_security_db.sql crea la base; el parámetro JDBC createDatabaseIfNotExist también permite su creación automática, y ddl-auto=update genera o ajusta las tablas del modelo.

Entidad de usuario y asociación JPA

```
@Entity
@Table(name = "users", uniqueConstraints = {
        @UniqueConstraint(name = "uk_users_username", columnNames = "username"),
        @UniqueConstraint(name = "uk_users_email", columnNames = "email")
})
public class AppUser {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 50)
    private String username;

    @Column(nullable = false, length = 120)
    private String email;

    @Column(nullable = false, length = 100)
    private String password;

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(name = "user_roles",
            joinColumns = @JoinColumn(name = "user_id"),
            inverseJoinColumns = @JoinColumn(name = "role_id"),
            uniqueConstraints = @UniqueConstraint(name = "uk_user_role", columnNames = {"user_id", "role_id"}))
    private Set<Role> roles = new HashSet<>();
```

Las rutas JDBC, el usuario y la contraseña de MySQL se configuran en src/main/resources/application.properties. La configuración de laboratorio usa root sin contraseña; si el servidor local difiere, se pueden definir DB_USERNAME y DB_PASSWORD.

Autenticación y reglas de acceso

SecurityFilterChain desactiva CSRF para esta API REST sin sesión, responde con JSON para los errores de seguridad y asigna permisos según la ruta. La aplicación permite registro e inicio de sesión públicos; el contenido de prueba restringe cada endpoint a los roles requeridos.

Cadena de seguridad y autorización por ruta

```
@Configuration
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    AuthenticationManager authenticationManager(AuthenticationConfiguration configuration) throws Exception {
        return configuration.getAuthenticationManager();
    }

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http, JwtAuthFilter jwtAuthFilter,
                                            ObjectMapper objectMapper) throws Exception {
        http.csrf(csrf -> csrf.disable())
                .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
                .exceptionHandling(exceptions -> exceptions
                        .authenticationEntryPoint((request, response, exception) -> writeError(
                                response, objectMapper, HttpServletResponse.SC_UNAUTHORIZED,
                                "Se requiere un token válido para acceder."))
                        .accessDeniedHandler((request, response, exception) -> writeError(
                                response, objectMapper, HttpServletResponse.SC_FORBIDDEN,
                                "El usuario autenticado no tiene permiso para esta ruta.")))
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers("/api/auth/**", "/api/test/all").permitAll()
                        .requestMatchers("/api/test/admin").hasRole("ADMIN")
                        .requestMatchers("/api/test/mod").hasRole("MODERATOR")
                        .requestMatchers("/api/test/user").hasAnyRole("USER", "ADMIN")
                        .anyRequest().authenticated())
                .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
```

BCryptPasswordEncoder se utiliza tanto al registrar usuarios como al autenticar. AuthenticationManager valida las credenciales a través de UserDetailsServiceImpl, que consulta la cuenta persistida y entrega sus autoridades a Spring Security.

Creación de cuentas iniciales

DataInitializer crea los tres roles y las cuentas de demostración solo cuando todavía no existen. El formulario público asigna siempre USER y rechaza solicitudes que intenten asignar ADMIN o MODERATOR, evitando la elevación de privilegios durante el registro.

Carga inicial idempotente de roles y usuarios

```
    @Transactional
    void initialize(RoleRepository roles, UserRepository users, PasswordEncoder encoder, String[][] accounts) {
        for (String roleName : List.of("USER", "ADMIN", "MODERATOR")) {
            if (roles.findByName(roleName).isEmpty()) {
                roles.save(new Role(roleName));
            }
        }
        for (String[] account : accounts) {
            String username = account[0];
            String email = account[1];
            String password = account[2];
            String roleName = account[3];
            if (!users.existsByEmailIgnoreCase(email)) {
                Role role = roles.findByName(roleName).orElseThrow();
                AppUser user = new AppUser(username, email, encoder.encode(password));
                user.addRole(role);
                users.save(user);
            }
        }
    }
```

Las contraseñas de demostración se codifican antes de guardarse. Si una cuenta ya existe, el inicializador no la sobrescribe ni restablece su clave.

Emisión y validación del token JWT

Al iniciar sesión, JwtUtils firma un token con HS256, guarda el correo como subject y fija la expiración configurada. En las peticiones siguientes, JwtAuthFilter lee Authorization: Bearer, valida firma y vencimiento, recupera el usuario y establece su Authentication en SecurityContextHolder.

Firma, lectura y validación del JWT

```
private Key signingKey() {
    return Keys.hmacShaKeyFor(Decoders.BASE64.decode(secretBase64));
}

public String generateJwtToken(Authentication authentication) {
    UserDetailsImpl principal = (UserDetailsImpl) authentication.getPrincipal();
    Date issuedAt = new Date();
    return Jwts.builder()
            .setSubject(principal.getUsername())
            .setIssuedAt(issuedAt)
            .setExpiration(new Date(issuedAt.getTime() + expirationMs))
            .signWith(signingKey(), SignatureAlgorithm.HS256)
            .compact();
}

public String getEmailFromJwtToken(String token) {
    return Jwts.parserBuilder().setSigningKey(signingKey()).build()
            .parseClaimsJws(token).getBody().getSubject();
}

public boolean validateJwtToken(String token) {
    try {
        Jwts.parserBuilder().setSigningKey(signingKey()).build().parseClaimsJws(token);
        return true;
    } catch (JwtException | IllegalArgumentException exception) {
        return false;
    }
}
```

Filtro que procesa el token por petición

```
@Override
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
                                FilterChain filterChain) throws ServletException, IOException {
    String header = request.getHeader("Authorization");
    if (header != null && header.startsWith("Bearer ")) {
        String token = header.substring(7);
        if (jwtUtils.validateJwtToken(token)
                && SecurityContextHolder.getContext().getAuthentication() == null) {
            String email = jwtUtils.getEmailFromJwtToken(token);
            UserDetails userDetails = userDetailsService.loadUserByUsername(email);
            var authentication = new UsernamePasswordAuthenticationToken(
                    userDetails, null, userDetails.getAuthorities());
            authentication.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
            SecurityContextHolder.getContext().setAuthentication(authentication);
        }
    }
    filterChain.doFilter(request, response);
}
```

La política es stateless: el servidor no mantiene una sesión HTTP. Un token ausente o inválido no autentica la solicitud (401); un usuario autenticado que carece del rol requerido recibe 403. La clave JWT provista en properties es solo para el laboratorio; JWT_SECRET_BASE64 permite reemplazarla por una clave propia de al menos 32 bytes.

Secuencia de autenticación

1. Login	Postman envía email y contraseña a /api/auth/signin.
2. Validación	AuthenticationManager compara la contraseña con el hash BCrypt guardado.
3. Emisión	JwtUtils devuelve un accessToken firmado y con vencimiento.
4. Solicitud	El cliente envía Authorization: Bearer a una ruta protegida.
5. Autorización	El filtro establece la identidad; Spring Security decide según el rol.

Registro, inicio de sesión y endpoints

AuthController valida campos, evita duplicados, cifra contraseñas y asigna el rol USER en el registro. En el login delega la autenticación y devuelve el token con el tipo Bearer, los datos básicos de la cuenta y las autoridades.

Registro público y autenticación

```
@PostMapping("/signup")
@Transactional
public ResponseEntity<MessageResponse> register(@Valid @RequestBody SignupRequest request) {
    if (request.roles() != null && request.roles().stream()
            .anyMatch(role -> !"USER".equalsIgnoreCase(role))) {
        return ResponseEntity.badRequest().body(new MessageResponse(
                "El registro público solo permite el rol USER."));
    }
    if (users.existsByUsernameIgnoreCase(request.username().trim())) {
        return ResponseEntity.status(HttpStatus.CONFLICT)
                .body(new MessageResponse("El nombre de usuario ya está registrado."));
    }
    if (users.existsByEmailIgnoreCase(request.email().trim())) {
        return ResponseEntity.status(HttpStatus.CONFLICT)
                .body(new MessageResponse("El correo ya está registrado."));
    }
    Role userRole = roles.findByName("USER").orElseThrow();
    AppUser user = new AppUser(request.username().trim(), request.email().trim().toLowerCase(),
            passwordEncoder.encode(request.password()));
    user.addRole(userRole);
    users.save(user);
    return ResponseEntity.status(HttpStatus.CREATED)
            .body(new MessageResponse("Usuario registrado correctamente con rol USER."));
}

@PostMapping("/signin")
public ResponseEntity<?> authenticate(@Valid @RequestBody LoginRequest request) {
    try {
        Authentication authentication = authenticationManager.authenticate(
                new UsernamePasswordAuthenticationToken(request.email(), request.password()));
        UserDetailsImpl principal = (UserDetailsImpl) authentication.getPrincipal();
        List<String> authorities = principal.getAuthorities().stream()
                .map(authority -> authority.getAuthority()).toList();
        return ResponseEntity.ok(new JwtResponse(jwtUtils.generateJwtToken(authentication), "Bearer",
                principal.getId(), principal.getDisplayName(), principal.getEmail(), authorities));
    } catch (AuthenticationException exception) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
                .body(new MessageResponse("Correo o contraseña incorrectos."));
    }
}
```

Endpoints principales

Método	Ruta	Acceso y respuesta
POST	/api/auth/signup	Público; 201 si registra. 400 si pide rol elevado; 409 si username/email ya existen.
POST	/api/auth/signin	Público; 200 con accessToken. 401 si credenciales incorrectas.
GET	/api/test/all	Público; 200.
GET	/api/test/user	USER o ADMIN; 401 sin token, 403 sin permiso.
GET	/api/test/admin	ADMIN; 401 sin token, 403 sin rol.
GET	/api/test/mod	MODERATOR; 401 sin token, 403 sin rol.

Ejemplo de inicio de sesión

Solicitud JSON	{"email":"user@test.com","password":"User@S09"}
Uso del token	Authorization: Bearer

El cuerpo de la respuesta de login incluye accessToken, tokenType, id, username, email y roles. El token completo se omite de este informe; se genera de nuevo en cada inicio de sesión.

Pruebas de autorización en Postman

La colección postman/Lab09-JWT.postman_collection.json reúne las solicitudes de registro, inicio de sesión y pruebas por rol. Los inicios de sesión de las cuentas de demostración guardan el token en variables de colección para reutilizarlo en las siguientes solicitudes.

Escenario	Token / perfil	Resultado esperado
Endpoint público	Sin token → /api/test/all	200 OK
Ruta protegida sin credencial	Sin token → /api/test/user	401 Unauthorized
Token modificado	Token inválido → ruta protegida	401 Unauthorized
Usuario en ruta de usuario	USER → /api/test/user	200 OK
Usuario intenta ruta admin	USER → /api/test/admin	403 Forbidden
Administrador en ruta admin	ADMIN → /api/test/admin	200 OK
Login con clave incorrecta	Credenciales inválidas	401 Unauthorized
Registro solicita ADMIN	Campo roles manipulado	400 Bad Request

Cuentas de demostración

Correo	Contraseña local	Rol
admin@test.com	Admin@S09	ROLE_ADMIN
user@test.com	User@S09	ROLE_USER
moderator@test.com	Mod@S09	ROLE_MODERATOR

Estas credenciales son de laboratorio. El inicializador solo las crea cuando el correo no existe; cambiar las variables de entorno no reemplaza las claves almacenadas para usuarios previamente creados.

Estructura del proyecto y responsabilidades

Paquete	Responsabilidad
config	SecurityFilterChain, codificador BCrypt y carga inicial de datos.
controller	Endpoints REST de registro/login y rutas de prueba por rol.
dto	Objetos de entrada y respuesta; validación del contrato HTTP.
jwt	Creación y validación de tokens y filtro de autenticación Bearer.
model	Entidades JPA AppUser y Role, incluida la asociación muchos a muchos.
repository	Consultas de usuarios y roles mediante Spring Data JPA.
service	Adaptación del usuario y sus roles al sistema de autenticación.

Conclusiones

• Separé la autenticación, que confirma quién realiza la petición, de la autorización, que determina a qué recursos puede acceder cada rol.

• El token firmado permite identificar al usuario en cada solicitud sin mantener una sesión en el servidor.

• BCrypt almacena hashes de las contraseñas en MySQL en lugar de texto legible.

• La tabla intermedia user_roles permite asignar uno o más roles a cada usuario y mantener la relación de forma relacional.

• Los escenarios de Postman documentan respuestas permitidas y denegadas para verificar rutas públicas, tokens inválidos y permisos por rol.

Observaciones

• La base security_db es independiente de las bases usadas en las semanas anteriores.

• createDatabaseIfNotExist requiere que la cuenta de MySQL tenga permiso para crear bases; Hibernate administra las tablas después de conectar.

• El acceso público solo crea cuentas USER; ADMIN y MODERATOR se reservan para la carga local inicial.

• Un token válido puede expirar; en ese caso el usuario debe autenticarse otra vez para obtener uno nuevo.

• El secreto JWT de ejemplo y las contraseñas de demostración se deben reemplazar antes de cualquier uso fuera del entorno local.

Con este laboratorio quedó implementada una API independiente con persistencia JPA, autenticación JWT sin estado y control de rutas por roles. La documentación del código, el modelo, las credenciales locales y las pruebas de Postman permiten reproducir el flujo desde XAMPP y Visual Studio Code.